

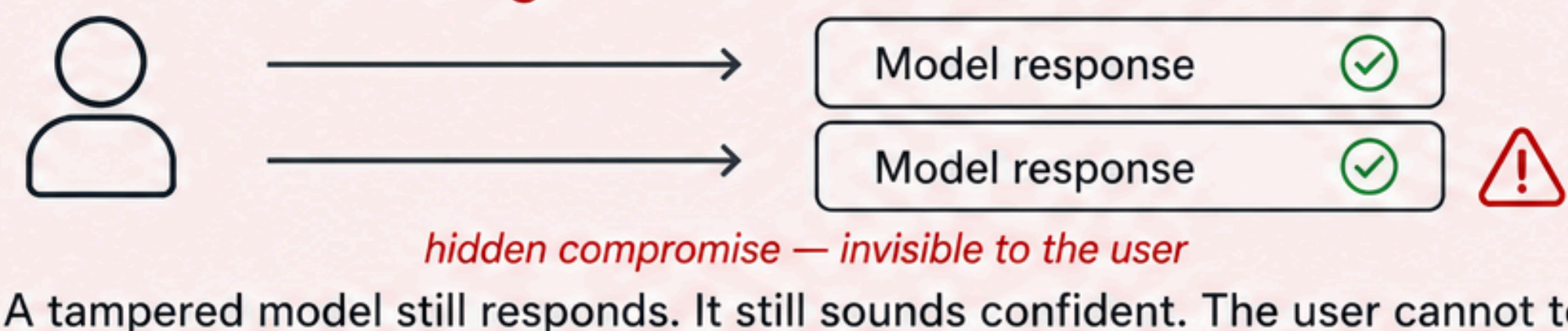
Large Language Models in the Real World



What an Attacker Can Do With a Tampered Model

- Inject a backdoor that triggers on a specific phrase or input
 - Bias outputs silently in targeted scenarios without detection
 - Neutralize safety filters and override deployer constraints
 - Leak sensitive information from user prompts under specific conditions
- The attacker does not need to replace the model — small targeted changes are enough.

No Warning. No Error. No Visible Difference.



Active Integrity Attestation Is Required

- Verify files on disk
 - Verify weights in memory
 - Verify behavior at runtime
- Detecting tampering requires a system that continuously monitors the model — not just at deployment, but throughout operation.

ADVERSARY MODEL



The attacker IS the model process

(unprivileged Linux user account)

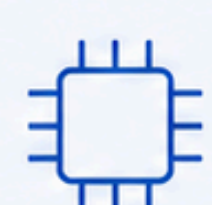
- Can read and write model files on disk
- Can modify model weights in memory
- Can attach debuggers to own processes
- Can inject code via LD_PRELOAD or Python



What the attacker CANNOT do

- Cannot become root or another user account
- Cannot modify kernel-enforced policies
- Cannot access or forge the verifier's signing keys

EXISTING APPROACHES



TPM + IMA

(Trusted Platform Module + Integrity Measurement Architecture)

Records a cryptographic fingerprint of every file loaded by the system. If any file changes after deployment, tampering is detected.



AttestLLM

(TEE + Activation Watermarking)

Periodically sends secret test inputs to the model and checks responses against expected patterns. Detects in-memory tampering even if files are untouched.

	TPM + IMA	AttestLLM
Detects disk-level tampering (A1, A2)	✓	✗
Detects in-memory tampering (A3)	✗	✓
Detects runtime parameter abuse (A4)	✗	⚡
Works on commodity hardware	✓	✓



Neither approach alone covers the full attack surface — a layered design is required.

KERNEL / ROOT DOMAIN (High Privilege, Trusted)

IMA (Integrity Measurement Architecture)

Kernel-level component that intercepts file loads and measures integrity

5. extends PCR
(Platform Configuration Register)

TPM (Trusted Platform Module)

ATTACKER DOMAIN / MODEL PROCESS (Untrusted)

A1
Model file replacement

A2
Tokenizer tampering

STORAGE (SSD)

Model Weights File
(e.g., model.safetensors)
101101 01011

Tokenizer File
(e.g., tokenizer.json)
110010 10100

Config Files
(e.g., config.json)
100111 01101

CPU

TEE (Trusted Execution Environment)

Watermark Trigger Inputs
(Secret Test Inputs)

Expected Activation Patterns
(Reference Signatures)

2. loaded into memory
(from storage)

3. trigger inputs
(periodically sent)

4. activations observed
(return path)

6. PCR quote
(Platform Configuration Register quote)

RAM (System Memory)

Model Weights
(in memory)

Tokenizer
(in memory)

Inference State
(in memory)

A3
In-memory weight tampering

USER DOMAIN / VERIFIER (Unprivileged, Trusted)

A4
Runtime parameter manipulation

POLICY ENGINE (Policy Enforcement)

Validates inference call parameters against an authorized policy

BASLINE FILE (Known-Good Hashes)

Stores known-good hashes of model files (tokenizer, weights, config, etc.)

VERIFIER DAEMON (Attestarain Verifier)

Receives PCR quote from TPM and verifies against known-good baseline

ABBREVIATIONS

LLM — Large Language Model

CPU — Central Processing Unit

TPM — Trusted Platform Module

IMA — Integrity Measurement Architecture

TEE — Trusted Execution Environment

PCR — Platform Configuration Register

SSD — Solid State Drive

RAM — Random Access Memory

DATA FLOW LEGEND

- Intra-domain data flow
- Cross-domain data flow (Trust boundary crossing)
- TEE → Model process (Periodic test inputs)
- Model process → TEE (Observed activations)
- Attack vector (caught by...)

ATTACKS & DETECTION

- A1 Model file replacement → Caught by TPM + IMA
- A2 Tokenizer tampering → Caught by TPM + IMA
- A3 In-memory weight tampering → Caught by AttestLLM / TEE
- A4 Runtime parameter manipulation → Caught by Policy Engine

LIMITATIONS AND FUTURE WORK

Current Limitations

- A5 (execution-state tampering) is identified but not yet defended
- Root compromise disables all defenses — root is assumed trusted
- Supply chain attacks are out of scope — baseline must be initialized from a trusted source
- Physical attacks (cold boot, hardware tampering) require hardware-level protections beyond this design

Future Work

- Design a defense for A5 through logit distribution monitoring or KV cache integrity checks
- Extend the framework to cloud deployments and multi-user scenarios
- Implement and empirically evaluate detection rates across all attack vectors

Conclusion

- A layered design combining TPM+IMA and AttestLLM covers attacks A1 through A4 across disk, memory, and runtime layers.
- Neither approach alone is sufficient — the combination closes coverage gaps that each individual approach leaves open.
- Future work includes defending against execution-state tampering (A5) and extending the design to cloud and multi-user deployment scenarios.

References

- [1] R. Sailer, X. Zhang, T. Jaeger, and L. van Doorn, "Design and Implementation of a TCG-based Integrity Measurement Architecture," in Proc. 13th USENIX Security Symposium, San Diego, CA, USA, Aug. 2004, pp. 223–238.
- [2] R. Zhang, Y. Zhao, N. Javidnia, M. Zheng, and F. Koushanfar, "AttestLLM: Efficient Attestation Framework for Billion-scale On-device LLMs," arXiv preprint arXiv:2509.06326, 2025.

Poster graphics generated with assistance from Claude (Anthropic) and ChatGPT (OpenAI).